

sim4comp report

Giuseppe Pianeti

2026-02-10

The Simulation for Comparison project is a simulation study that evaluates the performance of multiple machine learning (ML) models and techniques for binary classification across different scenarios. In this report, we first present the project structure and the simulation settings, then briefly describe the models and methodological choices, and finally discuss the results.

Project Structure

The project is organized into three main R scripts: `gendata_function.R`, `main.R`, and `charts.R`. The first one contains the Data Generating Process (DGP) function. Although originally developed for different purposes, it is well suited to the needs of this simulation study, as it generates multivariate data structures with flexible specifications. In particular, it produces a multivariate distribution $\mathbf{X} = (X_1, X_2, \dots, X_p)$ with pre-specified marginal distributions, conditional on the class membership variable Y , which represents the response variable of interest. Using a Gaussian copula combined with the inverse transform sampling method, the function generates samples from the conditional distribution $X_1, X_2, \dots, X_p \mid Y$, with a user-defined correlation structure. The main simulation parameters include the number of predictors `p`, the number of classes `c`, the sample size `n`, the marginal distributions `distrib` and their parameters `param`, the correlation structure `corr`, and the class prior probabilities `priors`. The input syntax is documented in the corresponding file. In our work we consider just two predictors (`p=2`) for visualization purposes and two output classes, consisting of a binary classification problem (`c=2`), with equal marginal probability for each class; therefore, the `prior` argument is omitted. The remaining inputs of the `gendata()` function vary across the different scenarios, which are described later.

The `main.R` script is used to simulate the datasets for each scenario and to train and evaluate the models. Specifically, for each scenario, 100 training datasets of size 100 are generated, resulting in 100 fitted models per method. These models are then evaluated on a large test dataset consisting of 1,000 instances. The machine learning methods considered in the analysis include K-nearest neighbors (KNN), Logistic Regression (LR), Linear and Quadratic Discriminant Analysis (LDA and QDA), Naive Bayes (NB), Random Forest (RF), Gradient Boosting Machines (GBM), and Bayesian Additive Regression Trees (BART). For each scenario and each method, predictions are obtained on the test dataset, and the corresponding error rates are computed, yielding an empirical distribution of performance. These distributions are then compared across models to assess their relative accuracy under different data-generating conditions.

Finally, the distributions are visualized using boxplots in the third script, `charts.R`. As noted in the `README.md`, the `final.results` RDS file included in the repository already contains the results obtained with a fixed random seed. Alternatively, running the `charts.R` script alone will reproduce the results and generate the corresponding outputs in the `outputs` folder.

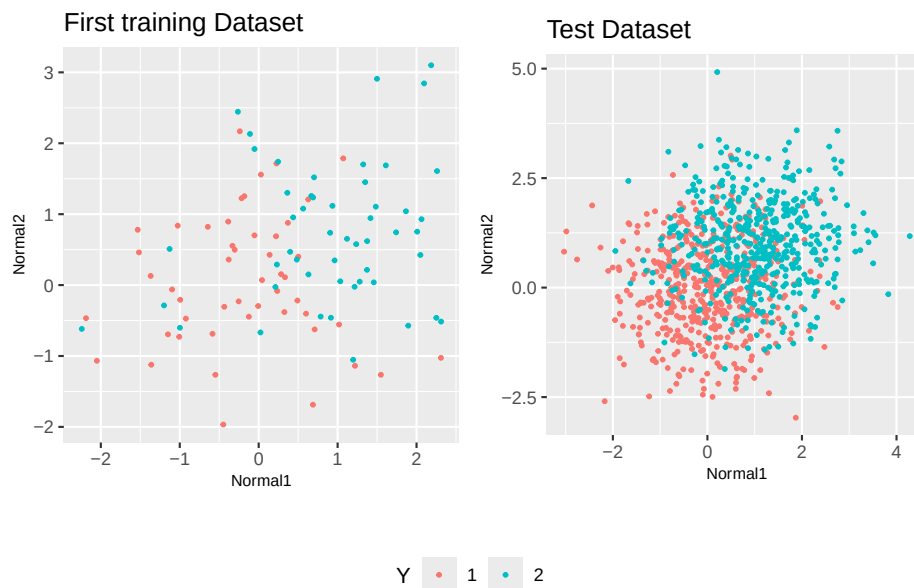
Scenarios

We define six main scenarios, categorized into linear and non-linear settings. In the linear case, the correlation structure among variables is homoscedastic across classes (this generates linear decision boundary), while in

the non-linear case it is heteroscedastic, i.e., the correlation structure varies between classes, except for the 6th scenario that is described below (in this case the decision boundary is non-linear). We now describe the key features of the linear scenarios.

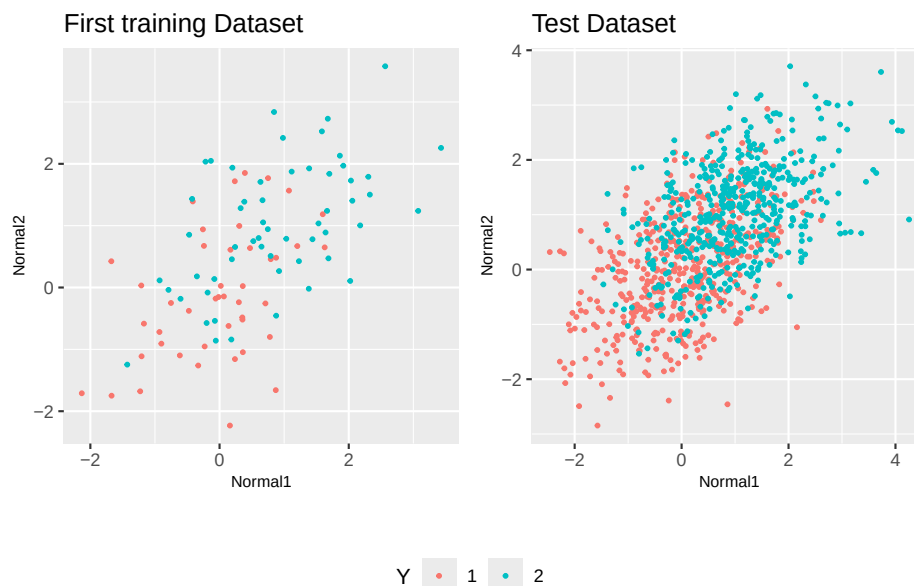
Scenario 1

Uncorrelated gaussian predictors with different means in each class.



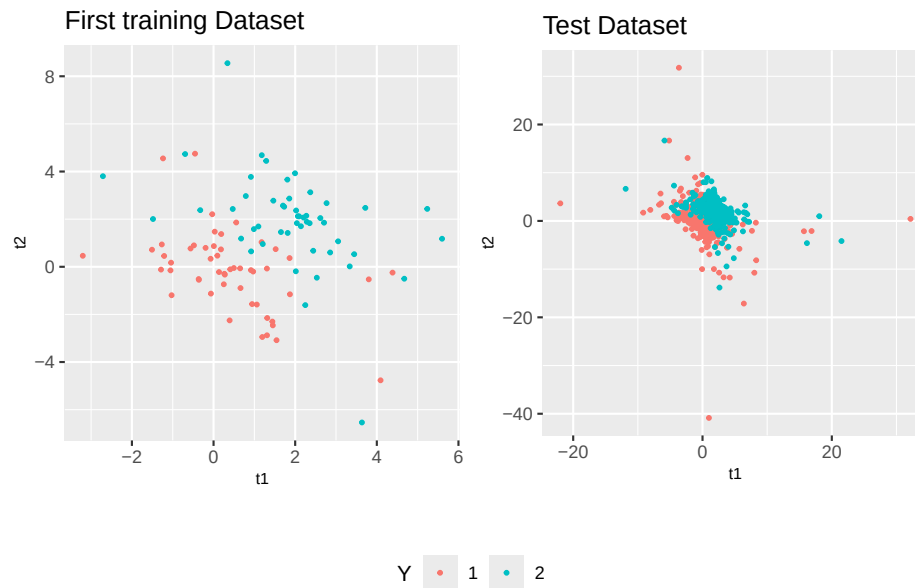
Scenario 2

Correlated gaussian predictors with different means in each class.



Scenario 3

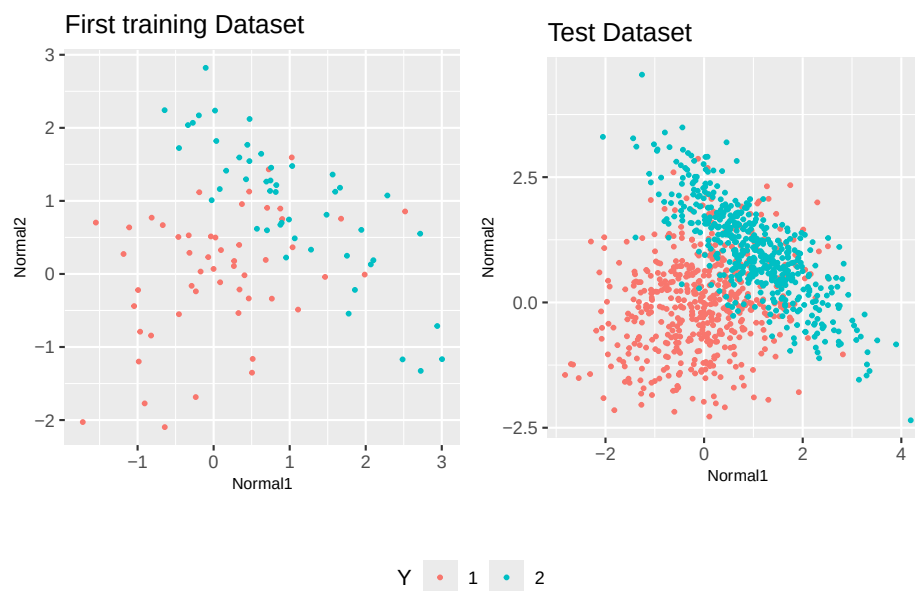
Correlated t-distributed predictors with 2 degrees of freedom and different means in each class.



For the non-linear case, instead, we have:

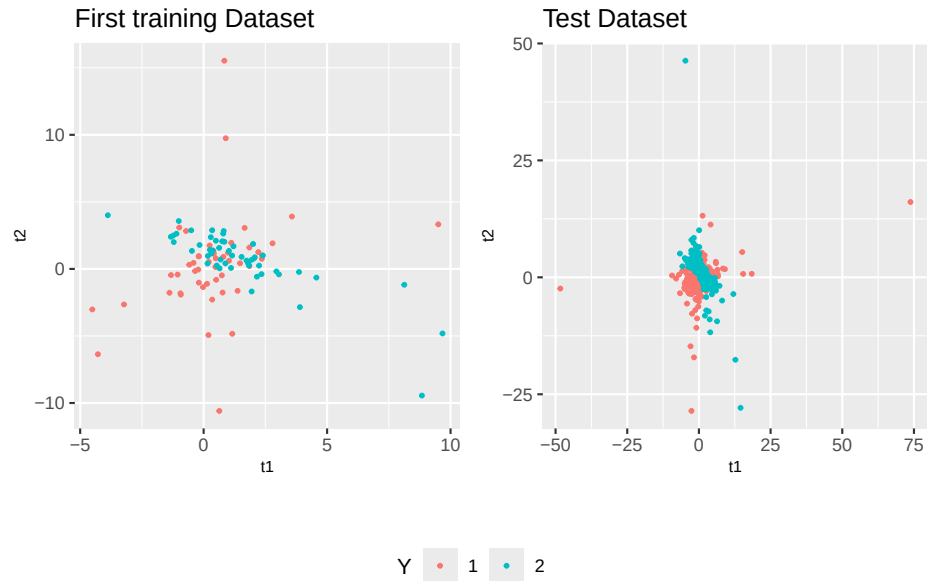
Scenario 4

Correlated gaussian predictors with different means and correlation in each class.



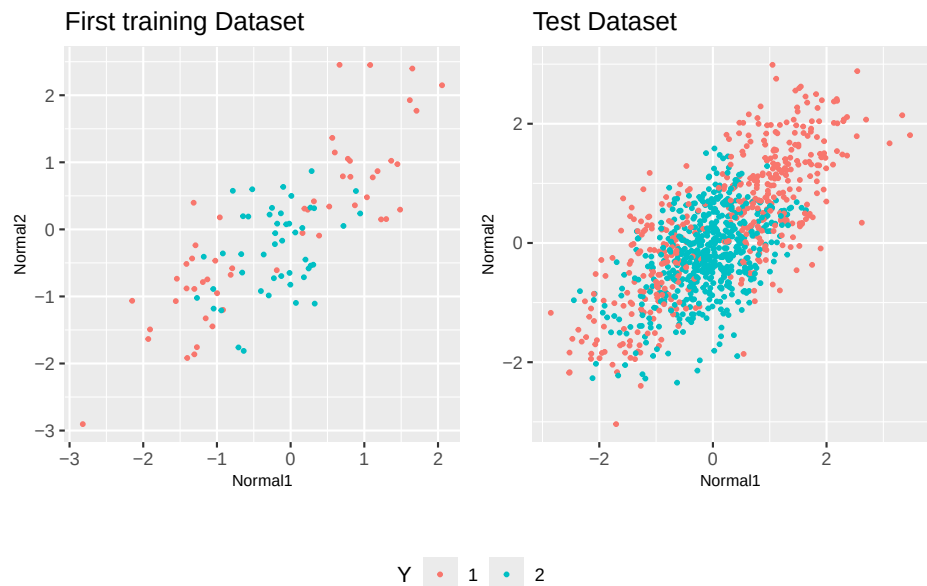
Scenario 5

Correlated t-distributed predictors with different means and correlation in each class.



Scenario 6

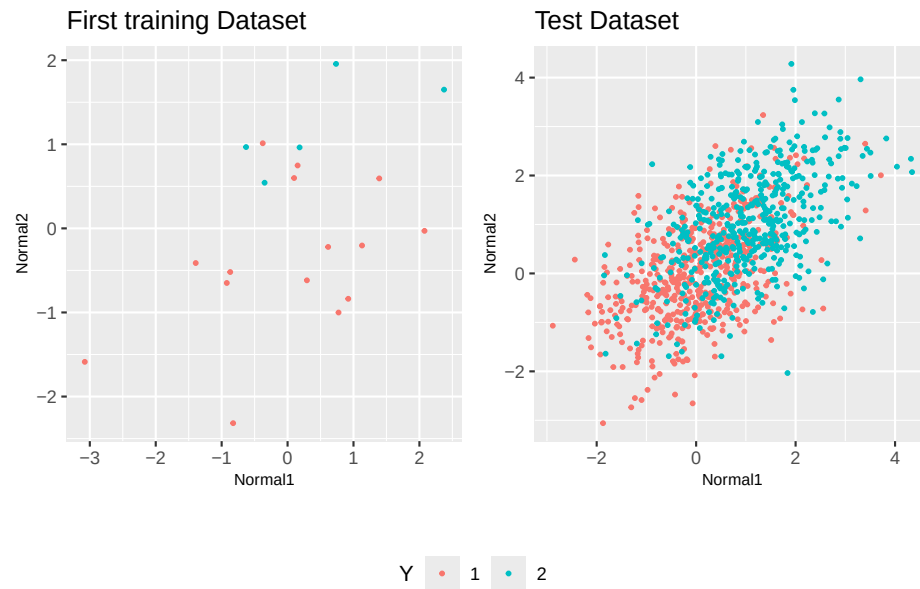
Correlated gaussian predictors whose response corresponds to a non-linear function of themselves. This final scenario is not generated using 'gendata()', as the structure of the function does not support it. Instead, we employ a custom-built function that generates a quantitative response as a function of the predictors, which is subsequently converted into two categorical classes using its expected value as a threshold.



To further investigate the parametric versus non-parametric assumptions of the models, as well as their behavior across different sample sizes, we introduce two additional scenarios with a smaller sample size ($n = 20$).

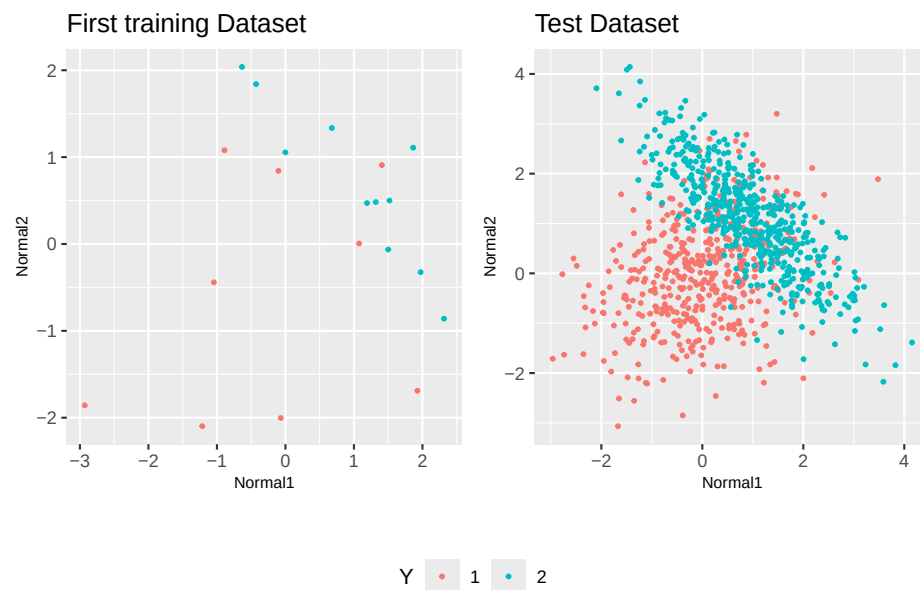
Scenario 7

Correlated gaussian predictors with homoschedastic correlation.



Scenario 8

Correlated gaussian predictors with eteroschedastic correlation.



Fitting Procedures

The only methods that require hyperparameter tuning are K-nearest neighbors (K-nn), Boosting, Random Forest, and BART, while the remaining models are fitted using standard R functions with default procedures. BART requires relatively limited tuning, as its regularization mechanism helps prevent overfitting. Therefore, we consider only two parameter configurations, depending on the sample size, as reported below:

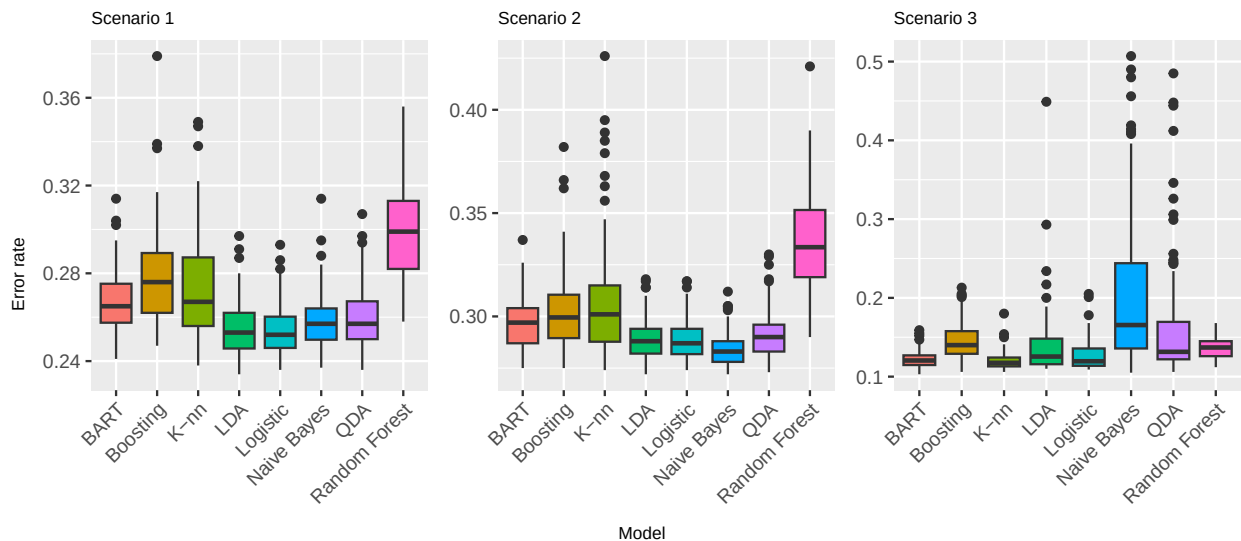
BART parameters	n= 20	n= 100
Fitted Trees	50	200
Burn-in iterations	500	200
Total iterations	1000	1200

For KNN, the optimal number of neighbors is selected via 10-fold cross-validation. A similar approach is adopted for Boosting. In the Boosting framework, we use a relatively large number of decision stumps (trees with a single split) to reduce computational cost during cross-validation. Specifically, we consider a grid ranging from 20 to 1000 trees in increments of 20 (49 candidate values), resulting in a total of 49,000 fitted models. The shrinkage parameter is fixed at 0.01. This choice is actually reasonable in presence of small number of instances (from 20 to 100), although the imposed limits on the tuning grid may restrict the model’s performance. Therefore, the Boosting results should be interpreted with this limitation in mind. For Random Forest, the default values for the mtry and nodesize parameters are retained.

Final Results

In this section, we present the results of the simulation (from the `final.results` file). Although the boxplots are organized according to the scenario categories introduced earlier, it is important to note that comparing the median performance of the models across different scenarios would be misleading. This is because the regions of class overlap—where classification is inherently uncertain—vary substantially between scenarios. As a result, the minimum achievable error rate, as defined by the Bayes optimal classifier, differs accordingly.

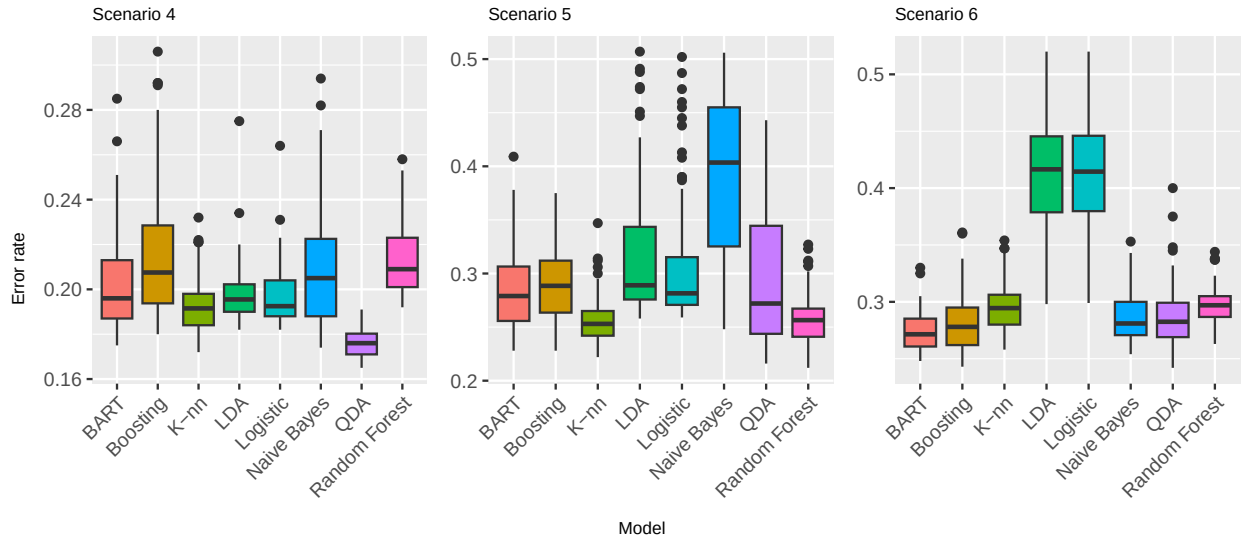
Linear Scenarios



A consistent pattern emerges across the first two scenarios for all evaluated methods. As anticipated, LDA and Logistic Regression perform exceptionally well in homoscedastic settings—where the decision boundary

is linear—regardless of whether the predictors are correlated. QDA and Naive Bayes also show strong results, with the latter achieving the highest performance in the second scenario. Conversely, more complex methods such as Boosting, BART, and Random Forests, which rely on orthogonal partitions of the predictor space, likely struggle to approximate this linear boundary, leading to poorer outcomes. This trend shifts dramatically in the third scenario with the introduction of heavy tails: Naive Bayes, QDA, and LDA exhibit higher median test errors and significant variability, while BART, k-NN, and Random Forests emerge as the top performers. We can conclude that, in this context, the violation of parametric assumptions (specifically normality) renders the traditional models’ estimates unreliable.

Non-Linear Scenarios



In the fourth scenario, QDA significantly outperforms all other models. This is expected, as the data structure aligns perfectly with the model’s underlying assumptions, resulting in the lowest median error and minimal variance. However, in the fifth scenario, the introduction of heavy tails leads to much more volatile results: while some QDA iterations still achieve the minimum error rate, they exhibit substantial variability. Instead, non-parametric models—led by k-NN and Random Forest—take the lead. While BART/Boosting and LDA/Logistic Regression show comparable median error rates, the latter group suffers from significant performance swings. In the final scenario, characterized by a highly non-linear decision boundary, LDA and Logistic Regression struggle by attempting to impose a linear fit, leading to severe underfitting and high test error rates. Conversely, more flexible methods, particularly BART and Boosting, successfully capture the non-linearities and yield strong results.

Few Instances Scenarios

In the small-sample case, we replicate the settings of scenarios 2 and 4 but with a significantly reduced sample size to test the limits of model flexibility. We observe a clear upward trend in error rates compared to previous scenarios; furthermore, non-parametric models—such as k-NN and tree-based methods—struggle to compete with simpler models, as they typically require larger datasets to produce reliable estimates. In the seventh scenario, Naive Bayes emerges as the top performer. Its rigid assumptions regarding data structure help reduce estimator variance at the cost of some bias, leading to remarkably strong results. Conversely, in the final scenario, Naive Bayes performs surprisingly poorly, whereas LDA, Logistic Regression, and QDA yield much better results.

